Lab 12 report — One local Compose experiment

รายงาน Lab 12 — ทดลอง Compose ในเครื่องด้วยแอปเดียว

Student ID / รหัสนักศึกษา: 6788165

Name / ชื่อ: Pongsakorn Arayasak

OS and Docker context / ระบบปฏิบัติการและ Docker context: Windows, desktop-linux

Repository URL / URL repository: https://github.com/MUICT-Class/691-itcs258-691-itcs258-l10-ppngskrn.git

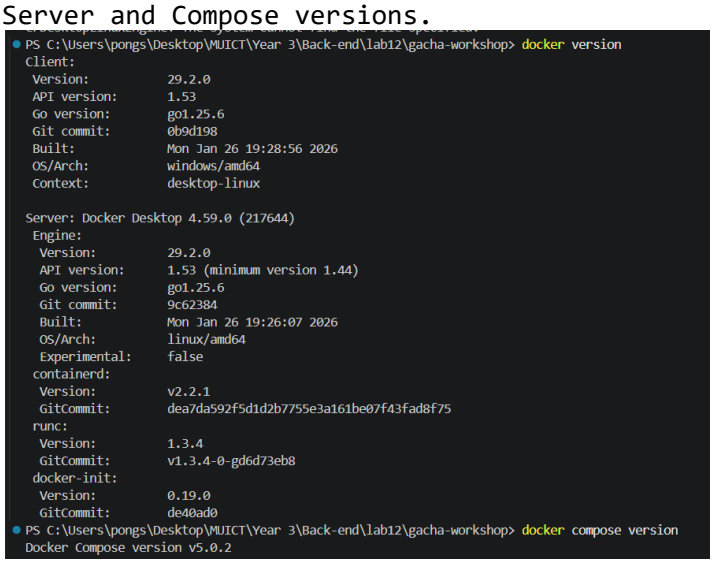
Dr. Pisit Praiwattana, your mentor in 2026

Complete required sections **01–04** using the gacha workshop. Use real commands/results and readable screenshots from **your own screen**. Export **`student_ID_lab12.PDF`** and submit one file to MyCourse by its posted deadline.

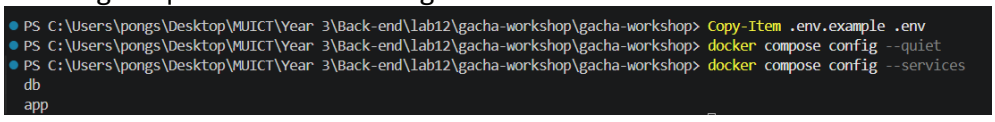
ทำส่วนบังคับ **01–04** ด้วย gacha workshop ใช้คำสั่ง/ผลจริงกับภาพอ่านได้จาก **หน้าจอตนเอง** ส่งออก **`student_ID_lab12.PDF`** แล้วส่งไฟล์เดียวใน MyCourse ตามกำหนด

01 · Folder, environment and readiness / โฟลเดอร์ environment และความพร้อม

- Docker Client/Server and Compose versions.



- Explain the folder roles: app source, Dockerfile, Compose files, `.env.example`, actual `.env`, SQL initialization.
- Show `config --quiet` and `config --services` results.



- Explain CLI interpolation versus container environment; show only sanitized configuration.

ตรวจเวอร์ชัน Engine/Compose อธิบายหน้าที่ไฟล์ แสดงผลตรวจ config และแยก interpolation ของ CLI ออกจาก environment ใน container ไม่แสดงข้อมูลลับจริง

Evidence and explanation / หลักฐานและคำอธิบาย:

Folder roles (what each file is for)

- App source (server.js, app.js, domain.js, init-db.js, openapi.json, public/test.html): the Express gacha API and the /test page. They run inside the app container.

- package.json / package-lock.json: the list of Node.js dependencies. The Dockerfile runs `npm ci --omit=dev`, which installs exactly the locked versions and skips dev-only packages.

- Dockerfile: the recipe for the app image only (node:24-bookworm-slim, copy the files, run as the non-root user `node`, start with `node server.js`).

- compose.yml: describes the whole stack: the two services (app and db), their environment, healthchecks, startup order and the pgdata volume.

- compose.override.yml: loaded automatically on top of compose.yml. It only adds the local port mapping 127.0.0.1:8085:3000 for the app.

- .env.example: a safe template with example values that can be shared.

- .env: my real local values, created with `Copy-Item .env.example .env` and then STUDENT_ID changed to my own ID. It is not committed, and .dockerignore (`.env`, `.env.*`) keeps it out of the image build context.

- db/init.sql: SQL that creates the tables wallets, transactions and draws. It is mounted read-only into the db container for first-time initialisation, and a copy is also packaged into the app image.

CLI interpolation versus container environment

- Interpolation (done by the Compose CLI): every `docker compose ...` command reads `.env` and fills the `${...}` placeholders inside compose.yml before anything starts. For example `name: gacha-${STUDENT_ID:?Set STUDENT_ID in .env}` becomes `gacha-6788165`, which is why my containers are called gacha-6788165-app-1 / gacha-6788165-db-1 and my volume is gacha-6788165_pgdata. The `:?` makes Compose stop with an error message if the value is missing.

- Container environment: a container only receives the variables listed under `environment:` for its service. app gets PORT, PGHOST=db, PGPORT, PGDATABASE, PGUSER and PGPASSWORD; db gets POSTGRES_DB, POSTGRES_USER and POSTGRES_PASSWORD. STUDENT_ID is used only by Compose for naming and is not passed into either container. The `.env` file itself is not copied into the image.

- `docker compose config --quiet` prints nothing when the file resolves without errors. `docker compose config --services` lists the services, db and app, as in my screenshot.

- Sanitized configuration: I did not paste the full `docker compose config` output because it prints the resolved values, including the database password. My .env has this shape:

```
STUDENT_ID=6788165
POSTGRES_DB=gacha
POSTGRES_USER=gacha
POSTGRES_PASSWORD=********
```

02 · Compose keys and startup / Key และการเริ่มระบบ

- Explain the real `services`, `build`, `image`, `ports`, `environment`, `depends_on`, `healthcheck` and `volumes` settings.
- Show image build, stack startup, `ps` and useful logs.



```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\lab12\gacha-workshop\gacha-workshop> docker compose up -d --build --wait
[+] up 15/15
 ✓ Image postgres:18-bookworm Pulled
#1 [internal] load local bake definitions
#1 reading from stdin 631B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 352B 0.0s done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/node:24-bookworm-slim
#3 DONE 0.0s

#4 [internal] load .dockerignore
#4 transferring context:
#4 transferring context: 182B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 375B 0.0s done
#5 DONE 0.0s

#6 [1/7] FROM docker.io/library/node:24-bookworm-slim@sha256:c86ecead9e55a5210ac1727de38a4ca840bf03dcaf34dc9749addbf383b1920c
#6 resolve docker.io/library/node:24-bookworm-slim@sha256:c86ecead9e55a5210ac1727de38a4ca840bf03dcaf34dc9749addbf383b1920c 0.0s done
#6 DONE 0.1s

#7 [6/7] COPY --chown=node:node db ./db
#7 CACHED

#8 [5/7] COPY --chown=node:node app.js domain.js server.js init-db.js openapi.json ./
#8 CACHED

#9 [2/7] WORKDIR /app
#9 CACHED

#10 [3/7] COPY package*.json ./
#10 CACHED

#11 [4/7] RUN npm ci --omit=dev
#11 CACHED

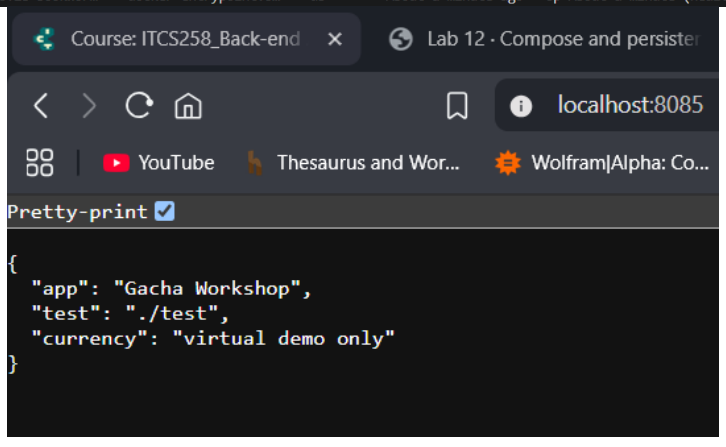
#12 [7/7] COPY --chown=node:node public ./public
#12 CACHED

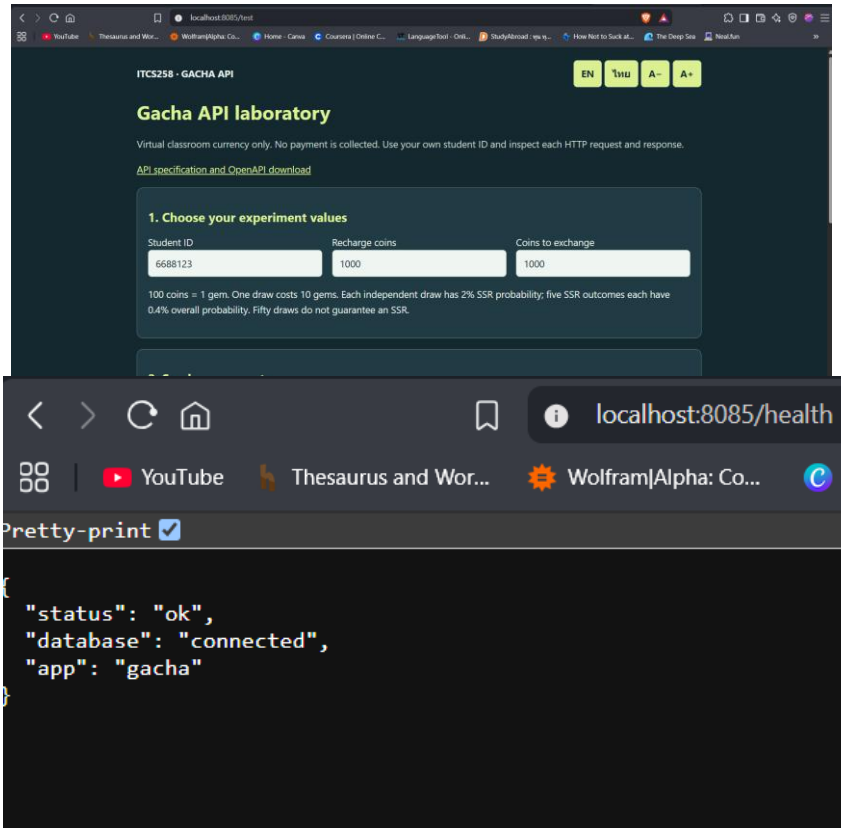
#13 exporting to image
#13 exporting layers 0.0s done
#13 exporting manifest sha256:a9f25adaedc0f38b4ba9f527096da3d19289e70ada8e7c35ef3b6da8b9019b09 done
#13 exporting config sha256:908b73213855ad87ae3dce03de216009b854c29982e21d89776dac871ff4dbed4 done
#13 exporting attestation manifest sha256:b317bf7c060e589932d8418e1bce9f277097b675e47bf0578da477596a575fe0 0.0s done
#13 exporting manifest list sha256:a26dee97d2bdcf1590bf9a0561e089fd581f3496700bb572007bcf72989bb243 0.0s done
#13 naming to docker.io/library/gacha-6788165-app:latest
#13 naming to docker.io/library/gacha-6788165-app:latest done
#13 unpacking to docker.io/library/gacha-6788165-app:latest 0.0s done
#13 DONE 0.2s

[+] up 20/20g provenance for metadata file
 ✓ Image postgres:18-bookworm    Pulled
 ✓ Image gacha-6788165-app       Built
 ✓ Network gacha-6788165_default Created
 ✓ Volume gacha-6788165_pgdata   Created
 ✓ Container gacha-6788165-db-1  Healthy
 ✓ Container gacha-6788165-app-1 Healthy
```

- Show healthy app/database and actual `/health` output.

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\lab12\gacha-workshop\gacha-workshop> docker compose ps
NAME                IMAGE                COMMAND                  SERVICE   CREATED              STATUS                        PORTS
gacha-6788165-app-1   gacha-6788165-app    "docker-entrypoint.s…"   app       About a minute ago   Up About a minute (healthy)   127.0.0.1:8085->3000/tcp
gacha-6788165-db-1    postgres:18-bookworm "docker-entrypoint.s…"   db        About a minute ago   Up About a minute (healthy)   5432/tcp
```





- Draw laptop → app and app → DB with hostnames and ports.

อธิบาย key จริง ผล build/start/ps/log และ health วาดการเชื่อมต่อเครื่อง⟶app กับ app→DB พร้อมชื่อและพอร์ต

Evidence and explanation / หลักฐานและคำอธิบาย:

- **services:** two services, app and db. A service is the definition while a container is one running instance of it (gacha-6788165-app-1 and gacha-6788165-db-1).

- **build: . (app):** Compose builds the app image from the Dockerfile in this folder; the dot is the build context The result is the image gacha-6788165-app

- **image: postgres:18-bookworm (db):** no Dockerfile here; Compose pulls a ready-made PostgreSQL image ("Pulled" in my screenshot). `build` creates an image, `image` selects an existing one.

- **ports: "127.0.0.1:8085:3000" (app, from compose.override.yml):** host port 8085, reachable only from my own laptop (127.0.0.1), is forwarded to port 3000 inside the app container. `docker compose ps` shows 127.0.0.1:8085->3000/tcp for app, but only 5432/tcp for db, so the database is not reachable from the laptop; only the app can reach it.

- **environment:** app receives PORT=3000, PGHOST=db, PGPORT=5432 and the database name/user/password from .env. db receives POSTGRES_DB, POSTGRES_USER and POSTGRES_PASSWORD, which the postgres image uses to create the database and user the first time it starts.

- **depends_on (condition: service_healthy):** the app starts only after the db healthcheck passes, not merely after the db container has started. That is why db shows Healthy before app in my `up` output.

- **healthcheck:** db runs `pg_isready` (is PostgreSQL accepting connections?). app runs a small Node script that fetches http://127.0.0.1:3000/health and fails unless the answer is OK. Both use interval 5s, timeout 3s, retries 12, start_period 10s. "Running" only means the process exists; "healthy" means it actually works.

- **volumes:** the named volume pgdata is mounted at /var/lib/postgresql so the database files live outside the container, and ./db/init.sql is bind-mounted read-only at /docker-entrypoint-initdb.d/01-init.sql. The top-level `volumes: pgdata:` declares the volume; Compose created it as gacha-6788165_pgdata.

Commands and what my output shows

- **docker compose build app** only builds the image (7 steps, "Image gacha-6788165-app Built"); it starts nothing.

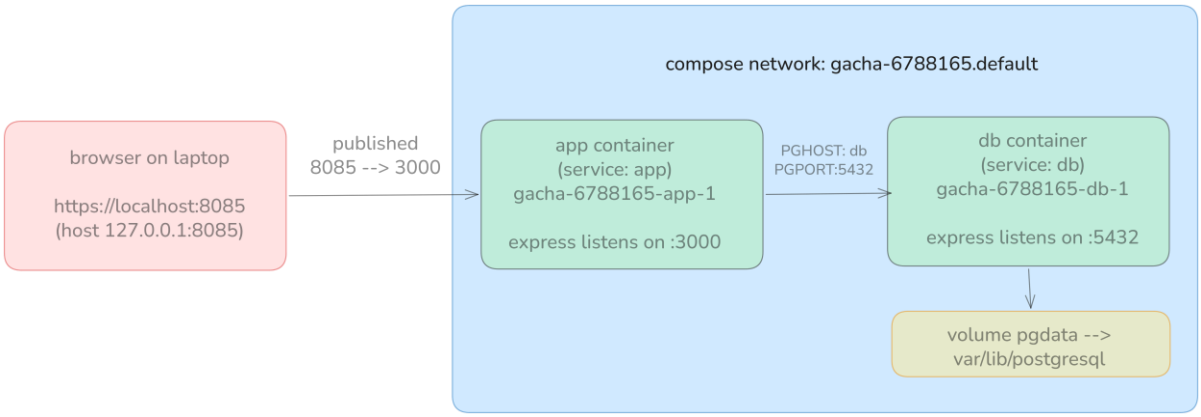
- **docker compose up -d --build --wait** rebuilds if needed (my second build was all CACHED), creates the network gacha-6788165_default and the volume gacha-6788165_pgdata, starts the containers in the background (-d) and waits until they are healthy (--wait). The output ends with db-1 Healthy and app-1 Healthy.

- **docker compose ps** shows both containers "Up ... (healthy)" and the port mapping 127.0.0.1:8085->3000/tcp.

- **/health** in the browser and with Invoke-RestMethod returned status ok, database connected, app gacha. "database: connected" shows that the app can really talk to PostgreSQL, which is stronger proof than just "container running".

- **Logs:** useful logs come from `docker compose logs --tail 30 app db`; when a service fails to start, the reason is in these logs.

Connection diagram (laptop → app, app → DB)



From the laptop I use localhost:8085. Inside the Compose network the app reaches the database by its service name, db, on port 5432. Inside the app

container "localhost" would mean the app itself, which is why PGHOST is db and not localhost.

03 · API and basic SQL / API และ SQL พื้นฐาน

- Use your own student ID in `/test` or equivalent API client.
- Show recharge1000 coins → exchange1000 coins →10gems → one draw → history.
- Record actual draw ID, result_name, rarity and created_at. COMMON is a valid result; SSR is not required.
- Show SQL connection, table list, SELECT matching the history and COUNT.
- Show the separate `-sql` wallet INSERT/SELECT practice.
- Explain 2% legendary probability and why a small experiment does not guarantee a fixed result count.

ใช้รหัสตนเองใน /test แสดงเดิม/แลก/สุ่ม/อ่าน history เก็บแถวจริง ผล COMMON ใช้ได้ ไม่บังคับ SSR ตรวจ SQL, SELECT/COUNT ที่ตรงกัน และฝึก INSERT ด้วย wallet -sql แยก อธิบายความน่าจะเป็น 2%

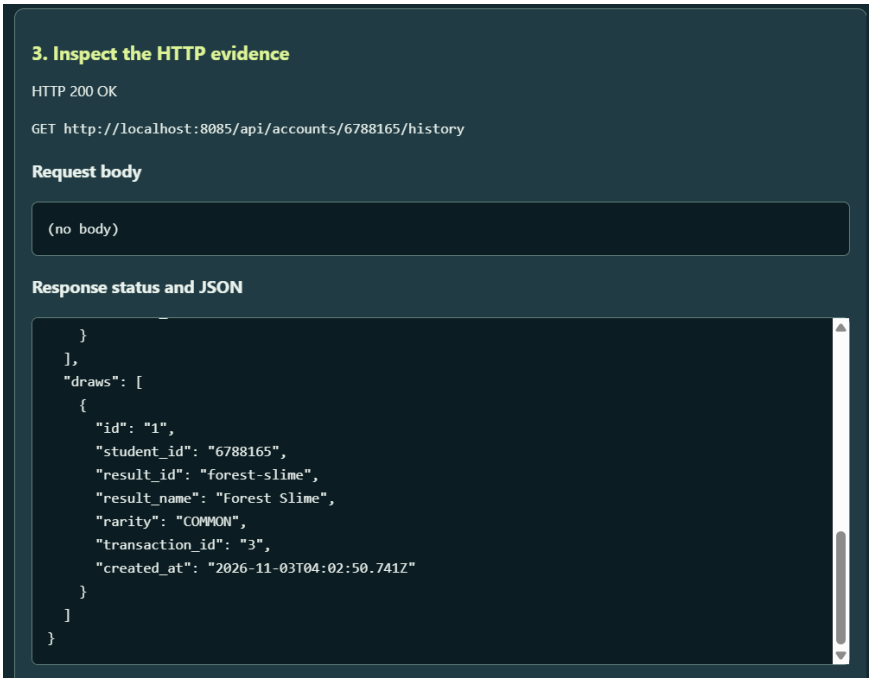




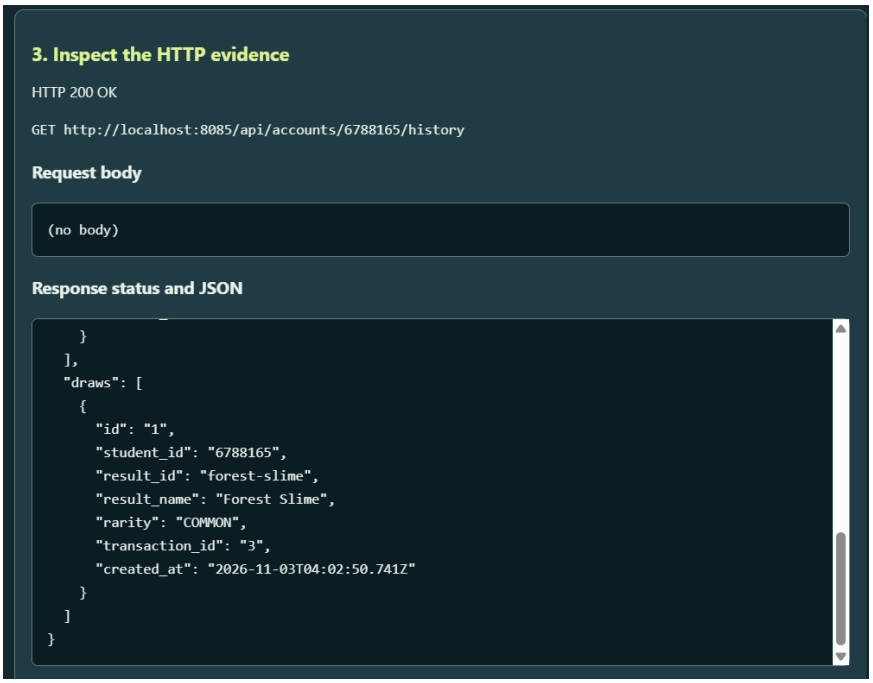
Original draw ID / ID draw เดิม: Original ID = 1, transaction id = 3



Original result + rarity / ผลและ rarity เดิม: Forest slime, COMMON



Original creation time / เวลาสร้างเดิม: 2026-11-03T04:02:50.741Z in the API; 2026-11-03 04:02:50.741071+00 in SQL



Evidence and explanation / หลักฐานและคำอธิบาย:

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\lab12\gacha-workshop\gacha-workshop> Invoke-RestMethod -Uri 'http://localhost:8085/health'

status database  app
------ --------  ---
ok     connected gacha


PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\lab12\gacha-workshop\gacha-workshop> Invoke-RestMethod -Uri 'http://localhost:8085/api/accounts/6788165/history'

transactions
------------
{@{id=3; student_id=6788165; kind=draw; coin_delta=0; gem_delta=-10; created_at=2026-11-03T04:02:50.741Z}, @{id=2; student_id=6788165; kind=exchange; coin_...
```

```
PS C:\Users\pongs\Desktop\MUICT\Year 3\Back-end\lab12\gacha-workshop\gacha-workshop> docker compose exec db psql -U gacha -d gacha
psql (18.6 (Debian 18.6-1.pgdg12+2))
Type "help" for help.

gacha=# \conninfo
         Connection Information
      Parameter       |        Value
----------------------+---------------------
 Database             | gacha
 Client User          | gacha
 Socket Directory     | /var/run/postgresql
 Server Port          | 5432
 Options              |
 Protocol Version     | 3.0
 Password Used        | false
 GSSAPI Authenticated | false
 Backend PID          | 2030
 SSL Connection       | false
 Superuser            | on
 Hot Standby          | off
(12 rows)

gacha=# \dt
              List of tables
 Schema |     Name     | Type  | Owner
--------+--------------+-------+-------
 public | draws        | table | gacha
 public | transactions | table | gacha
 public | wallets      | table | gacha
(3 rows)
```

```
gacha=# SELECT id,student_id, result_name, rarity, created_at FROM draws WHERE student_id = '6688123' ORDER BY id;
 id | student_id | result_name | rarity | created_at
----+------------+-------------+--------+------------
(0 rows)

gacha=# SELECT id,student_id, result_name, rarity, created_at FROM draws WHERE student_id = '6788165' ORDER BY id;
 id | student_id | result_name  | rarity |          created_at
----+------------+--------------+--------+-------------------------------
  1 | 6788165    | Forest Slime | COMMON | 2026-11-03 04:02:50.741071+00
(1 row)

gacha=# SELECT COUNT(*) AS draw_count FROM draws WHERE student_id = '6788165';
 draw_count
------------
          1
(1 row)
```

Note: In my first query I forgot to change the example ID to my own ID, so it returns with zero rows. I then ran the same query with my own ID 6788165.

```
gacha=# INSERT INTO wallets (student_id) VALUES ('6788165-sql') ON CONFLICT (student_id) DO NOTHING;
INSERT 0 1
gacha=# SELECT student_id, coins, gems FROM wallets WHERE student_id = '6788165-sql';
 student_id  | coins | gems
-------------+-------+------
 6788165-sql |     0 |    0
(1 row)
```

What the API sequence did (student ID 6788165)

- **Recharge 1000 coins** (HTTP 201): transaction 1, recharge, coin_delta +1000. Wallet: 1000 coins, 0 gems.

- **Exchange 1000 coins → 10 gems** (HTTP 201): transaction 2, exchange, coin_delta -1000, gem_delta +10. The rate is 100 coins = 1 gem. Wallet: 0 coins, 10 gems.

- **Draw once** (HTTP 201): transaction 3, draw, gem_delta -10 (one draw costs 10 gems). Wallet: 0 coins, 0 gems. A row is created in draws.

- **History** (HTTP 200): the draws list has id 1, result Forest Slime (forest-slime), rarity COMMON, transaction_id 3, created_at 2026-11-03T04:02:50.741Z.

The app never edits the balance directly. Every operation adds a row to the transactions ledger and the wallet balance follows from it, which is why the draw row points to transaction 3.

SQL evidence

- **docker compose exec db psql -U gacha -d gacha** runs the psql client inside the running db container. `\conninfo` shows database gacha, user gacha, port 5432 through the local socket; `\dt` lists the three tables draws, transactions and wallets.

- **SELECT** on draws for 6788165 returns 1 row: id 1, Forest Slime, COMMON, 2026-11-03 04:02:50.741071+00. This matches the API history (PostgreSQL prints microseconds and +00 = UTC; the JSON shows milliseconds with Z). COUNT(*) gives draw_count = 1.

- **Separate -sql wallet:** `INSERT INTO wallets (student_id) VALUES ('6788165-sql') ON CONFLICT (student_id) DO NOTHING;` printed INSERT 0 1 (one row inserted), and the SELECT shows 6788165-sql with 0 coins and 0 gems (the table defaults). student_id is the primary key, so ON CONFLICT (student_id) DO NOTHING means "if this wallet already exists, skip it without an error". I used a different ID so this practice does not change my proof account.

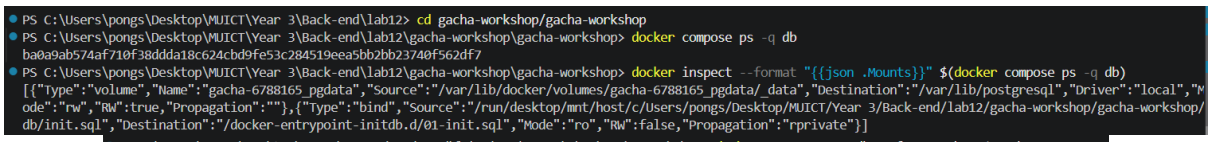
2% legendary probability

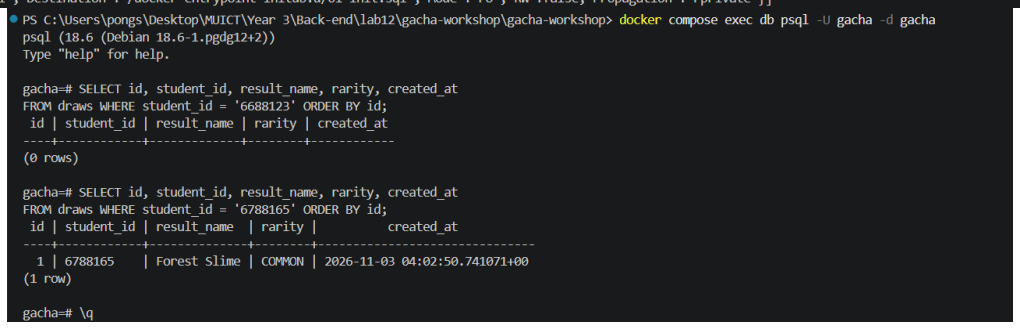
Every draw is independent and has a 2% chance (0.02) of a legendary/SSR result; the five SSR outcomes share it, so each is 0.4%. The chance of at least one SSR in n draws is 1 - 0.98^n: about 2% for 1 draw, 18.3% for 10 draws and 63.6% for 50 draws. It never reaches 100%, and previous draws do not change the next one, so no fixed number of draws guarantees an SSR. In my small experiment I made one draw and got COMMON, which is the normal outcome (98%). One draw is also far too small a sample to show a 2% rate.

04 · Persistence and bind mounts / ข้อมูลถาวรและ bind mount

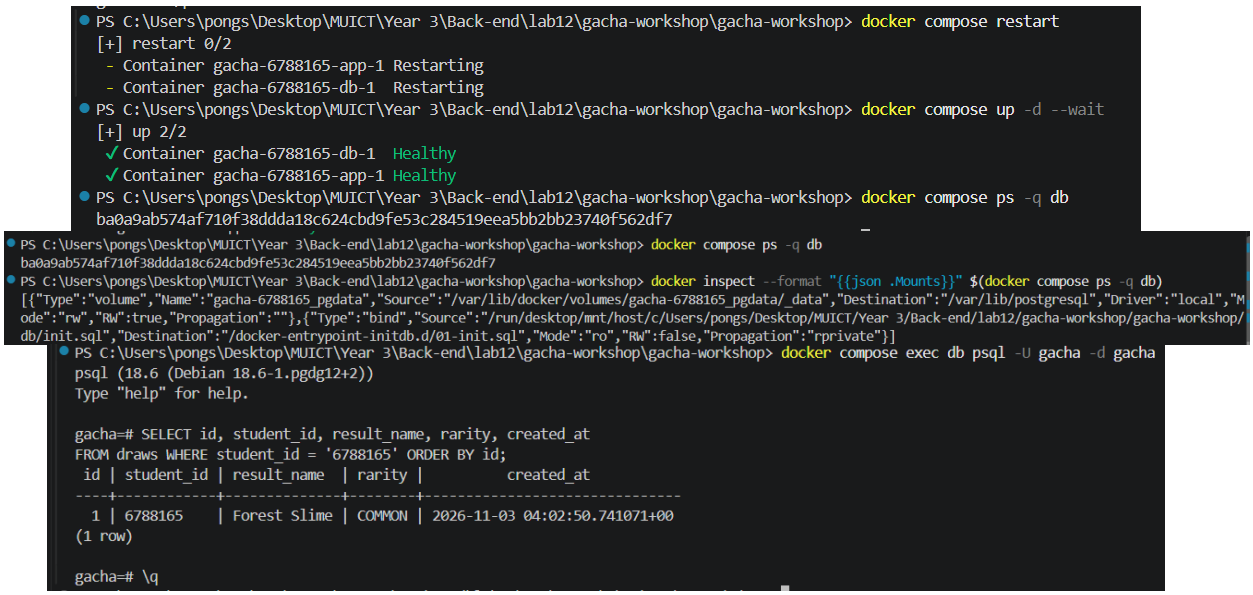
Stage / ช่วง	DB container ID	Named volume	Original draw ID	Result + rarity + created_at
Initial / เริ่มต้น	ba0a9ab574af710f38ddda18c624cbd9fe53c284519eea5bb2bb23740f562df7	gacha-6788165_pgdata	1	Forest Slime, COMMON, 2026-11-03 04:02:50.741071+00
Restart / หลัง restart	ba0a9ab574af710f38ddda18c624cbd9fe53c284519eea5bb2bb23740f562df7	gacha-6788165_pgdata	1	Forest Slime, COMMON, 2026-11-03 04:02:50.741071+00
Down /up / หลังสร้างใหม่	e6e9c03ccbda6ef766d3958b26d02049e0caf9773ff2542e04a2407dcd5a4db4	gacha-6788165_pgdata	1	Forest Slime, COMMON, 2026-11-03 04:02:50.741071+00

4.1 Initial

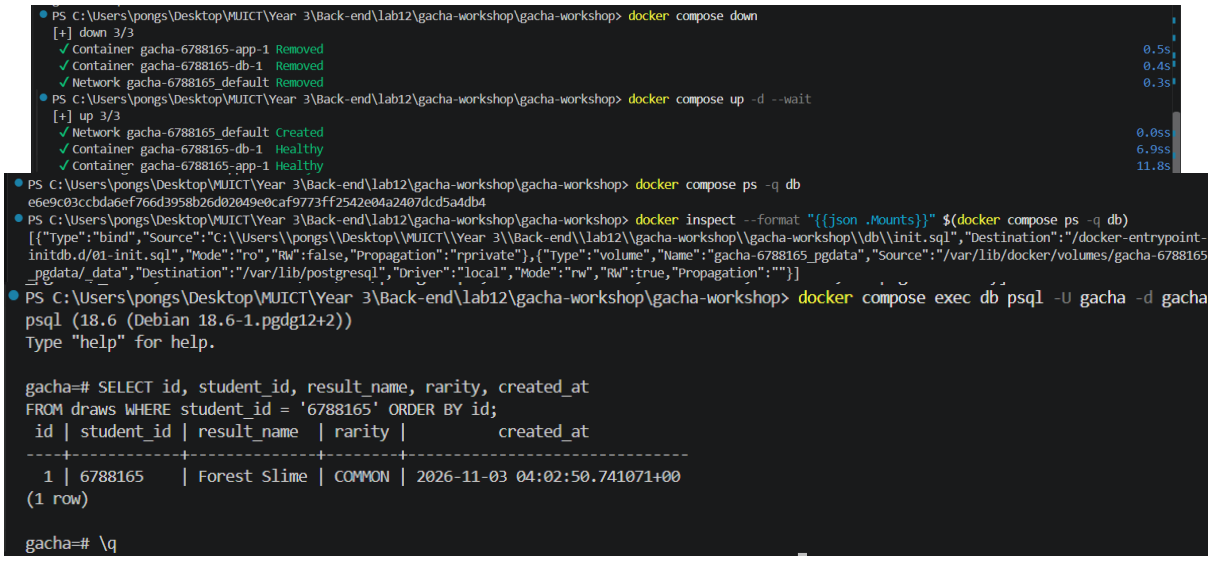




4.2 Restart



4.3 After down/up



- Show actual restart/down/up commands and the API history/SQL read after each.
- After down/up show a **new DB container ID**, **same volume**, and **unchanged original draw**, without drawing/reinserting it again.
- Explain named volume versus the read-only `db/init.sql` bind mount.
- Verify source path/type; explain one real or predicted bind-mount pitfall and fix.
- Distinguish the PostgreSQL entrypoint (empty data directory only) from app bootstrap (`server.js` executes its copied SQL at every app startup). Explain why current `CREATE TABLE IF NOT EXISTS` does not change existing columns, why schema changes need migrations, and what rebuilding copied SQL can affect.
- แยก entrypoint ของ PostgreSQL ที่รันเมื่อ data directory ว่าง ออกจาก bootstrap ของแอปที่ `server.js` รัน SQL ที่คัดลอกทุกครั้งที่เริ่ม อธิบายว่า `CREATE TABLE IF NOT EXISTS` ไม่แก้ column เดิม และการแก้ SQL ใน image ต้อง rebuild พร้อมวาง migration อย่างตั้งใจ

- Explain `down` versus `down -v`, and why volume persistence is not backup.

แสดงคำสั่งกับผลอ่านหลังแต่ละขั้น หลัง down/up ต้องเห็น ID container ใหม่ volume เดิม และ draw เดิมโดยไม่สุ่มซ้ำ อธิบาย named volume เทียบ bind mount แบบ read-only ตรวจ path/ชนิดไฟล์ อธิบายข้อผิดพลาดกับวิธีแก้ รวมถึง init/migration, down/down -v และ persistence/backup

Evidence and explanation / หลักฐานและคำอธิบาย:
Result of the persistence test

- Restart: `docker compose restart` restarted the existing containers, so the DB container ID stayed ba0a9ab574af...2df7 and the original draw (id 1, Forest Slime, COMMON) was still there.

- Down/up: `docker compose down` removed the two containers and then `docker compose up -d --wait` then created new containers. The DB container ID changed to e6e9c03ccbda...4db4, but the inspected volume is still gacha-6788165_pgdata and SQL returns the same draw id 1, Forest Slime, COMMON, 2026-11-03 04:02:50.741071+00. The data survived because it lives in the named volume, not in the container.

Named volume versus the read-only bind mount

- pgdata:/var/lib/postgresql (named volume): storage created and managed by Docker (Source /var/lib/docker/volumes/gacha-6788165_pgdata/_data, Type volume, rw). It holds the PostgreSQL data files, outlives any one container and is reused by the next db container.

- ./db/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro (bind mount): maps one specific file from my project folder into the container (Type bind, mode ro, RW false). I control it from the host, and :ro means the container can read it but cannot change it, which protects my input file.

- Source path and type check: in `docker inspect` the bind Source ends with db/init.sql (Docker Desktop prints it as /run/desktop/mnt/host/c/Users/.../db/init.sql the first time and as C:\Users\...\db\init.sql after down/up; it is the same host file) and the Destination is a file path, so it is a file-to-file mount. The bind source must already exist and be the expected type (file or folder).

Two different startup mechanisms

- PostgreSQL image entrypoint: runs the scripts in /docker-entrypoint-initdb.d (my 01-init.sql) only when the data directory is empty, i.e. the first time a new, empty volume is used. After down/up the pgdata volume was not empty, so init.sql was not run again and my data was untouched.

- App bootstrap: server.js runs the SQL that was copied into the app image (`COPY db ./db` in the Dockerfile) every time the app starts. The statements are CREATE TABLE IF NOT EXISTS, so they create missing tables but do not change tables or columns that already exist. Editing that copied SQL needs a rebuild of the app image, and real schema changes (for example adding a column) need proper migrations, not just editing init.sql.

down versus down -v, and why a volume is not a backup

- **docker compose down** removes the containers and the network but keeps named volumes, so the data stays.

- **docker compose down -v** also deletes the named volumes declared in the file, so the wallet and draw would be gone and the next `up` would start with an empty database (init.sql would run again). I did not use -v.

- **Not a backup:** a volume survives container removal and restart, but it is still one copy on one machine. It is lost with `down -v`, `docker volume rm`, a prune, resetting Docker Desktop or a disk failure, and it does not protect against bad data or a corrupted database. A real backup is a separate copy, for example a pg_dump file stored somewhere else.

In-room LA check / ตรวจ LA ในห้อง

Show your working experiment to the LA before leaving the room. Record a short summary of what you demonstrated and any correction you made.

แสดงการทดลองที่ทำงานได้ต่อ LA ก่อนออกจากห้อง บันทึกสั้น ๆ ว่าแสดงอะไรและแก้ไขสิ่งใด

Demonstration summary / สรุปสิ่งที่แสดง:

Optional practice / ฝึกเพิ่มเติมทางเลือก

If you chose another backend/framework, briefly show how the same pattern applies. This is optional and does not replace the single required gacha experiment.

หากเลือกฝึกแบ็กเอนด์/framework อื่น ให้สรุปการใช้รูปแบบเดียวกัน ส่วนนี้เป็นทางเลือกและไม่แทนการทดลอง gacha ที่บังคับ

Final check / ตรวจสอบก่อนส่ง

- All required parts 01–04 are present / ครบส่วนบังคับ 01–04
- Evidence comes from my own screen / ภาพจากหน้าจอตนเอง
- Original draw remains unchanged; no replacement was created / แถวเดิมไม่เปลี่ยนและไม่ได้สร้างแทน
- Repository URL and sanitized configuration included / มี URL repository และค่าตั้งไม่มีข้อมูลลับ
- Filename uses my actual ID: `student_ID_lab12.PDF` / ใช้รหัสจริงในชื่อไฟล์
- One PDF uploaded and checked for readability / ส่ง PDF เดียวและเปิดตรวจความชัดเจนแล้ว